

Competitive Exam Practice 7 — Answer Key

Doubly & Circular Lists, List-Backed Stack and Queue, and Polynomials

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 3, 2026

- Q1 (C) Circular doubly linked list.** To concatenate two lists in $O(1)$ you must reach both ends without a walk and be able to rewire the four boundary links; a circular doubly linked list stores a pointer to the last node whose `next` is the first node and whose `prev` is the previous last node, so both joins are constant-time splices. A singly or ordinary doubly linked list needs an $O(n)$ walk to reach the tail, and an array needs $O(n)$ copying/reallocation. (Local book answer key 3.10.6 = C; stem + options also on PracticePaper’s GATE CSE 1997 page.)
- Q2 (A) rear node.** Point `p` at the rear node. Then the front node is `p->next` (one link, $O(1)$), so `dequeue` removes at the front and `enqueue` inserts just after `p` and advances `p` to the new node — both constant time. Pointing at the front instead makes `enqueue` at the rear need an $O(n)$ walk to find the tail; two pointers would work but the question allows a single one, so (A) is the answer and (C) is false. (Local book answer key 3.10.12 = A.)
- Q3 (B) (II) only.** For a singly linked list to behave as a circular queue with $O(1)$ insert/delete, the ring must be closed from the rear back to the front: the rear node’s `next` points to the front node, so appending after `REAR` and advancing `REAR` keep `FRONT` reachable. Statement (I) points the front node forward at the rear, which does not make the rear reach the front and is not required. (Local book answer key 3.12.10 = B.)
- Q4 (B) $\Theta(1)$, $\Theta(n)$.** Inserting at the head is a constant-time front insert ($O(1)$). Deleting at the tail of a *non-circular* singly linked list requires the tail’s predecessor to redirect the tail’s `next` to `NULL`; with only a tail pointer there is no backward link, so the predecessor must be found by walking from the head — $\Theta(n)$ worst case. Hence `enqueue` is $\Theta(1)$ and `dequeue` is $\Theta(n)$. (Local book answer key 3.12.11 = B; all four options on GeeksforGeeks.)
- Q5 (D) `SLLdel` is $O(n)$ and `DLLdel` is $O(1)$.** In a doubly linked list a node reaches both neighbours through `prev` and `next`, so `DLLdel` unlinks it in $O(1)$ given a pointer to it. In a singly linked list the same node cannot reach its predecessor, so unless it is the first node (or the copy-next trick is used, which fails for the last node) deleting it requires an $O(n)$ walk to find the node whose `next` points at it. (Local book answer key 3.10.20 = D; all four options on GeeksforGeeks; GATEQA lists answer D.)
- Q6 (A) The list becomes `head` $\leftrightarrow$ 7 $\leftrightarrow$ 42; the two boundary links are rewired in $O(1)$ and the delete is $O(1)$ once `p` is known.** With `p` at the 19-node, `p->prev` is the 7-node and `p->next` is the 42-node, both in hand: set `7->next = p->next` (42) and `42->prev = p->prev` (7), then `free(p)`. (B) wrongly charges an $O(n)$ search the backward link eliminates; (C) denies the bypass ever happens; (D) updates only one link, leaving 42’s `prev` dangling — the classic half-invariant bug. Confirmed by trace (2026-10-03).

- Q7 (A).** Wire the new node forward and backward while `p->next` still names the old successor (14), repair 14's backward link, and only then overwrite `p->next` — i.e. `newNode->next = p->next`, then `newNode->prev = p`, then `p->next->prev = newNode`, and finally `p->next = newNode`. (B) overwrites `p->next` first, so `newNode->next` becomes a self-loop and 14 is lost. (C) and (D) overwrite `p->next` too early, so the later `p->next->prev = newNode` sets `newNode->prev` to itself and leaves 14's `prev` pointing at 8 — a broken two-link invariant. Confirmed by trace (2026-10-03).
- Q8 (B) It loops forever, cycling p, q, r, ...** In a ring no node's `next` is `NULL`, so the test `ptr != NULL` is never false and the walk never terminates. The correct traversal tests `ptr != head` (preferably with a `do--while` so the head is visited once). (A) would require a `NULL` terminator that a circle does not have; (C) misreads the loop condition as checked once; (D) is not what the loop does. Confirmed by trace (2026-10-03).
- Q9 (A) $A + B = 10x^3 + 5x^2 + 9$, one merge pass, $O(n + m)$; the cancelled x term is not stored.** Merge by descending exponent: exp 3 gives $6 + 4 = 10$; exp 2 exists only in B , so $5x^2$ copies through; exp 1 gives $2 + (-2) = 0$, and a zero-coefficient term is never emitted; exp 0 gives 9. Each source node is visited once, so the cost is linear. (B) stores a forbidden zero term; (C) confuses addition with the $O(n \cdot m)$ multiplication; (D) wrongly keeps $4x$. Confirmed by trace (2026-10-03).
- Q10 (A) push front-inserts 28 giving $\text{head} \rightarrow 28 \rightarrow 7 \rightarrow 14 \rightarrow 21$; `pop()` returns 28, leaving top 7; both $O(1)$ worst case.** The stack top is `head`, so `push` is Week 6's front-insert and `pop` is front-delete — each rewrites `head` and one link, both constant time. (B) appends at the tail and mis-states which end pops; (C) mislabels the LIFO discipline (a stack returns the newest, 28, not the oldest); (D) is false: no walk is needed at `head`. Confirmed by trace (2026-10-03).